Is a Cached Token Worth It? Measuring the Energy Discount of Prefix Caching in LLM Serving

Midterm Report

Aagam Sachin Bothara
Department of Computer Science, University of Illinois Chicago
aboth@uic.edu

TABLE I
Progress against proposal milestones

Milestone (proposal)	Status at Nov 1
Harness, cache controls, noise floor	done
Hit/miss verification per request	done
Zero-hit control (cache on, no reuse)	done
$D(L)$ sweep, Qwen-7B, 1-token out	done
Full grid (out 128/512, conc. 4/16)	done: 4 of 4 cells
PrefixBench-style vLLM reproduction	moved to Nov 2–8

I. Project Type and Status

This is a research project (individual). The question is unchanged: how much GPU energy does reusing a cached prefix actually save in SGLang [3], and does an accounting model with an explicit cached-token cost predict energy better than cache-unaware, pricing-inspired, and FLOP-informed baselines? Table I compares progress against the proposal milestones. The main change since the proposal is a precise definition of the experimental conditions (Section II), written in response to the feedback that the cache-on versus cache-off comparison was unclear.

II. Clarifying the Experimental Conditions

The proposal used "cache on" and "cache off" loosely. Those words hide three separate things: whether the radix cache is enabled, whether a request's prefix is actually found in it, and side effects of the flag itself. I now separate them into three conditions (Table II). In all of them the normal per-request KV cache used during decoding stays on; only cross-request prefix reuse changes.

Primary contrast: Hit vs. Miss. Both run on the same server process with the same flags and the same token counts. The only difference is whether the prefix is found in the cache, so their energy difference is the value of reuse and nothing else. I redefine the discount as

$$D(L) = \frac{E_{\text{miss}}(L) - E_{\text{hit}}(L)}{E_{\text{miss}}(L) - E_{\text{miss}}(0)}, \tag{1}$$

where E is mean active GPU energy per request, and suffix length, output length, and concurrency are held fixed.

TABLE II
Measurement conditions. L = shared-prefix length; last column is the expected cached_tokens per request.

Name	Radix cache	Prefix contents	Expected
Off	disabled	identical across requests	0
Miss	enabled	first token replaced by a per-request unique token, so no request matches another	0
Hit	enabled	identical; warmed by one excluded request	L

$E_{\text{miss}}(0)$ is measured with the prefix removed. I also report energy saved per reused token, $(E_{\text{miss}}(L) - E_{\text{hit}}(L))/L$.

Secondary contrast: Miss vs. Off. This is the zero-hit control from the proposal. It measures what enabling the cache costs when nothing is reused, $\Delta_{\text{on}}(L) = E_{\text{miss}}(L) - E_{\text{off}}(L)$, from tree insertion, matching, and eviction. If Δ_{on} is indistinguishable from zero, the "turn caching on" saving equals the reuse saving. If it is not, I report both. Zine et al. [2] saw energy differences from the prefix-cache flag on a workload with no repeated prefixes, which is exactly the effect this control isolates.

Confounders held fixed. (1) Scheduling policy. When the radix cache is disabled, SGLang silently replaces cache-aware policies such as LPM with FCFS (schedule_policy.py, _validate_and_adjust_policy). Without care, Off vs. Hit would compare two schedulers. I pin --schedule-policy fcfs in every condition. (2) Exact token counts. Requests are sent as token IDs to /generate, with no chat template, so prefix and suffix lengths are exact and no template tokens are accidentally shared. Each suffix also starts with a per-request unique token; with purely random tokens, two requests in a window can share a first token by chance and create a spurious 1-token hit. (3) Fixed work. ignore_eos with fixed max_new_tokens, greedy decoding, fixed chunked-prefill size, page size 1, and CUDA graphs on. (4) Cache state. /flush_cache before every window, then one excluded warm-up request in every condition, so all three pay the same first-request cost; in Hit this request also inserts the prefix. Its energy is recorded separately.

Verification. Every response's

meta_info.cached_tokens is logged. A window is accepted only if every request matches its condition's expected count in Table II. Across all 273 A100 windows (25,915 Hit requests and 32,110 Miss and Off requests), every Hit request reported exactly L cached tokens and every Miss and Off request reported 0, so no window was rejected.

III. Measurement Setup as Built

Platform. All Chameleon Cloud GPUs were reserved through November, so the main study runs on a rented RunPod A100 80 GB SXM4, Secure Cloud, a whole GPU per pod, driver 595.91.07, CUDA 13.2, SGLang 0.5.21 (pinned), with the same pod configuration in every session. Before any run, I checked that NVML's energy counter agrees with its instantaneous power reading at idle (87.11 W vs. 87.52 W). I cannot lock clocks or the power limit on a rented pod, so I log both per window and repeat a fixed reference window at the start of every session to detect drift between sessions and physical GPUs. Model: Qwen2.5-7B-Instruct, BF16. RunPod does not offer the A30, so an A40 (48 GB) serves as the second GPU. A short pilot on the A40 ran at its 300 W power cap in every window, so it is used only for spot checks.

Harness. A closed-loop load generator holds c requests in flight and runs windows of at least 40 requests and at least 30 s, so short 1-token windows are not dominated by counter resolution. The harness reads NVML's cumulative energy counter directly at window boundaries. (My llmtrace tool samples power and only supports vLLM, so it is not used for the SGLang runs.) Active energy subtracts idle power (measured for 60 s with the model loaded, before each block) times window duration. Run order is randomized across conditions and L. Each window logs SM clock, temperature, power limit, and any other GPU process.

IV. Preliminary Results

Noise floor. Three repeats of an identical Miss window (L=2048, 1-token output, c=1) gave a coefficient of variation of 0.8% in active energy per request. This sets the smallest saving I can resolve with three repeats. The same reference window gave 45.50 J per request on the first A100 and 45.90 J on the second, a 0.9% difference, about the size of the noise floor.

Zero-hit control. Δ_{on} was +0.34 J per request (95% CI [−0.39, +0.83]) at L=8192, i.e., within the noise floor. Across all L in the 1-token sweep, Δ_{on} stayed within +0.9% of E_{off}, and within +2.7% across the full grid (usually within 1%). One caveat: Miss and Off need different server flags, so they run on two server processes, and a small Δ_{on} could partly be a restart effect.

Discount curve. Table III gives $D(L)$ with 1-token outputs, which isolates prefill. D rises from 0.986 at L=512 to 0.995 at L=8192, approaching 1; the saving is far outside the 0.8% noise floor at every L. With 1-token outputs, a pricing rule of $b = 0.1a$ predicts $D \approx 0.9$; the measured values are above this, since a cached token cost about 0.08 mJ against about 20 mJ for a recomputed one. Longer outputs are expected to lower D, because decoding still reads the cached prefix; those cells are complete (Table IV). D falls to 0.92–0.97 at 128 output tokens and 0.87–0.93 at 512 (for $L \geq 2048$), while the energy saved per reused token stays between 0.017 and 0.028 J in every cell.

TABLE III
Preliminary $D(L)$: Qwen2.5-7B, A100, 1-token output, c=1. Mean of 3 windows; 95% bootstrap CI.

L	E_{miss} (J)	E_{hit} (J)	$D(L)$	J saved / token
512	15.55	6.73	0.986 [0.983, 0.988]	0.0172
1024	27.28	6.76	0.993 [0.992, 0.993]	0.0200
2048	45.87	6.91	0.992 [0.991, 0.993]	0.0190
4096	89.74	7.07	0.994 [0.994, 0.994]	0.0202
8192	172.27	7.35	0.995 [0.995, 0.996]	0.0201

TABLE IV
$D(L)$ across output length and concurrency: Qwen2.5-7B, A100, mean of 3 windows per condition. The out = 512, c=4 cell ran on a second A100 in a later session; its 95% CIs at $L \leq 1024$ are wide ([0.62, 0.98] and [0.91, 1.05]).

	out 1	out 128		out 512	
L	c=1	c=4	c=16	c=4	c=16
512	0.986	0.924	0.949	0.795	0.903
1024	0.993	0.918	0.968	0.973	0.921
2048	0.992	0.948	0.962	0.933	0.915
4096	0.994	0.949	0.966	0.865	0.903
8192	0.995	0.960	0.969	0.872	0.913

Where the saving comes from. The saving is mostly time, not power. In the 1-token sweep at L=8192, Miss and Off windows drew about 397 W of the A100's 400 W limit, with the SM clock pulled down to about 1260 MHz, while Hit drew about 363 W at the full 1410 MHz. Power fell by under 10%, but each request finished far sooner. At L=8192 the grid cells drew 356–395 W, and every logged clock was 1410 MHz.

Reproduction. Moved to Nov 2–8; I prioritized the condition controls above after the proposal feedback.

V. Changes from the Proposal

(1) The primary comparison is now Hit vs. Miss in a single server configuration, and the radix-off condition is a control rather than the baseline for D. (2) Requests are sent as raw token IDs. (3) The scheduling policy is pinned. (4) Windows have a minimum duration as well as a minimum request count. The research questions, models, baselines, and RQ2 accounting model (Eq. 2 of the proposal) are unchanged. (5) The platform moved from Chameleon Cloud to a RunPod A100 because every Chameleon GPU was reserved through November. Section III describes how I validated the rented GPU's energy counter and how I control for session-to-session drift. (6)

RunPod does not offer the A30, so the second-GPU checks move to an A40.

VI. Plan to December 6 and Risks

Nov 2–8: PrefixBench-style reproduction on vLLM; rerun Δ_{on} with Miss and Off alternating across several server restarts, so restart effects average out; repeat the out $= 512$, c=4 cell at $L \leq 1024$ to narrow its CIs. Nov 9–15: Llama-3.1-8B-Instruct on the A100; A40 checks at $L \in \{2048, 8192\}$. Nov 16–22: held-out workloads (ShareGPT, shared agent system prompts, LongBench RAG templates) with fixed-rate and Poisson arrivals; fit the four accounting variants on synthetic data only. Nov 23–26: evaluate RQ2 on held-out windows; release the harness on GitHub. Nov 27–Dec 3: presentation. Dec 4–6: final report.

Risks. (1) If Δ_{on} is non-zero, D and the "turn caching on" saving diverge; I will report both rather than pick one. (2) At high concurrency, unique Miss prefixes can fill the KV pool and trigger eviction. I flush between windows, and an eviction inside a Hit window would show up as a request with fewer than L cached tokens and reject the window. None has occurred so far; if one does at L=8192, I will reduce the window size. (3) The additive model may miss batching effects; I will report errors by load level instead of a single aggregate. (4) A rented pod may land on a different physical GPU each session; the per-session reference window lets me treat sessions as blocks in the analysis, and I will report it if drift exceeds the noise floor.

Use of AI Tools

Claude (Anthropic) was used to help structure this report, check SGLang's scheduling code for cache-dependent behavior, and edit language. All measurements and numbers come from my own runs.

References

[1] O. Shewale, D. Kumar, and D. K. Yadav, "PrefixBench-H100: Characterizing prefix reuse and time-to-first-token in H100 LLM serving," arXiv:2609.19657, 2026.

[2] N. Zine et al., "Attention to detail: Evaluating energy, performance, and accuracy trade-offs across vLLM configurations," arXiv:2607.09172v2, 2026.

[3] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in Proc. NeurIPS, vol. 37, 2024.